

// Jivops

Envoy Proxy

Guía completa — 20 lecciones

Fundamentos · Configuración dinámica xDS · Resiliencia y tráfico · Seguridad y operación

Contenido

Módulo 1: Fundamentos

- Lección 01 — Qué es Envoy Proxy
- Lección 02 — Listeners, filter chains y HTTP connection manager
- Lección 03 — Rutas y virtual hosts
- Lección 04 — Clusters y endpoints
- Lección 05 — Modelo de hilos y configuración bootstrap

Módulo 2: Configuración dinámica (xDS)

- Lección 06 — Configuración estática y dinámica
- Lección 07 — APIs xDS: LDS, RDS, CDS, EDS
- Lección 08 — ADS y consistencia de la configuración
- Lección 09 — Planos de control
- Lección 10 — Interfaz de administración y config_dump

Módulo 3: Resiliencia y tráfico

- Lección 11 — Balanceo de carga
- Lección 12 — Health checks y outlier detection
- Lección 13 — Circuit breaking
- Lección 14 — Reintentos y timeouts
- Lección 15 — Limitación de tasa

Módulo 4: Seguridad y operación

- Lección 16 — TLS y mTLS con SDS
- Lección 17 — Autenticación y autorización
- Lección 18 — Observabilidad
- Lección 19 — Extensibilidad: Lua y Wasm
- Lección 20 — Proyecto final: Envoy listo para producción

Módulo 1: Fundamentos

Lección 01 — Qué es Envoy Proxy

Un proxy de borde y de servicio de alto rendimiento (proyecto graduado de la CNCF), escrito en C++, que trabaja en capas 3/4 y 7 y se configura de forma dinámica mediante APIs; es la base de muchos service meshes y gateways.

Porque se configura dinámicamente por API, actúa bien como sidecar y aporta observabilidad, resiliencia y seguridad sin cambiar las aplicaciones.

Lección 02 — Listeners, filter chains y HTTP connection manager

Un listener escucha en una dirección y puerto; sus filter chains aplican filtros de red a la conexión, y para tráfico HTTP el filtro HTTP connection manager interpreta las peticiones, ejecuta los filtros HTTP y las enruta según la configuración de rutas.

Para procesar peticiones y respuestas antes de reenviarlas: autenticación, limitación, cabeceras, extensiones, etc.

Lección 03 — Rutas y virtual hosts

La configuración de rutas agrupa virtual hosts (seleccionados por dominio) con rutas que comparan prefijo, ruta exacta, expresión regular o cabeceras, y cada ruta indica el cluster de destino, con opciones como reescritura, timeouts y reintentos.

Una ruta con clusters ponderados (weighted clusters) que reparte el 95 % y el 5 % entre las versiones.

Lección 04 — Clusters y endpoints

Un grupo de endpoints (servidores de destino) a los que Envoy reenvía tráfico, con su política de balanceo, health checks, límites de circuito y configuración TLS; los endpoints pueden ser estáticos o descubiertos dinámicamente.

El algoritmo de balanceo, los health checks, los límites de circuito y la configuración TLS hacia los destinos.

Lección 05 — Modelo de hilos y configuración bootstrap

Un hilo principal que gestiona configuración y administración y varios hilos worker (por defecto uno por núcleo) que aceptan conexiones; cada conexión se queda en un único worker durante toda su vida, lo que evita cerrojos; el fichero bootstrap define la configuración inicial.

Evita compartir estado y usar cerrojos entre hilos, lo que reduce la contención y mejora el rendimiento.

Módulo 2: Configuración dinámica (xDS)

Lección 06 — Configuración estática y dinámica

La estática (todo en el bootstrap) sirve para casos simples o de prueba, pero cambiar algo exige recargar o reiniciar; la dinámica obtiene listeners, rutas, clusters y endpoints de un servidor de control mediante xDS y se actualiza en caliente sin reiniciar.

Descubre los endpoints y aplica cambios en caliente, sin reiniciar el proxy ni perder conexiones.

Lección 07 — APIs xDS: LDS, RDS, CDS, EDS

LDS los listeners, RDS las rutas, CDS los clusters y EDS los endpoints de cada cluster (SDS además los secretos como certificados); un servidor de control las entrega por gRPC o REST y Envoy las aplica.

Porque los endpoints cambian constantemente (escalado, reinicios, fallos), mientras que los clusters cambian poco.

Lección 08 — ADS y consistencia de la configuración

Para recibir todos los tipos de recurso xDS por un único stream y así poder ordenar las actualizaciones (por ejemplo, entregar primero clusters y endpoints y después las rutas que los usan), evitando enrutar hacia destinos que aún no existen.

Que una ruta apunte a un cluster que todavía no existe, lo que provocaría errores 503 temporales.

Lección 09 — Planos de control

El servidor que calcula y entrega la configuración xDS a los Envoy a partir de una fuente de verdad, como Kubernetes; ejemplos son Istio, Envoy Gateway o servidores propios hechos con go-control-plane.

Porque es muy extensa y propensa a errores; un plano de control la genera a partir de recursos más sencillos.

Lección 10 — Interfaz de administración y config_dump

Un servidor HTTP con endpoints como /stats, /clusters, /listeners, /ready y /config_dump para inspeccionar el estado y la configuración efectiva, y para algunas operaciones; no debe exponerse a redes no confiables.

Porque no tiene autenticación, revela configuración y estado interno y permite operaciones como cerrar o vaciar conexiones.

Módulo 3: Resiliencia y tráfico

Lección 11 — Balanceo de carga

Entre otros, round robin ponderado, least request (por defecto elige el mejor de dos endpoints al azar), random, ring hash y Maglev para afinidad por hash de cabecera o IP; también admite balanceo por zonas para reducir latencia y costes.

Cuando las peticiones duran de forma desigual: envía al endpoint con menos peticiones activas y evita sobrecargar los lentos.

Lección 12 — Health checks y outlier detection

Los health checks activos sondean a los endpoints de forma periódica, mientras que outlier detection es pasivo: observa los errores del tráfico real y expulsa temporalmente a los endpoints que fallan; suelen combinarse.

El activo detecta caídas aunque no haya tráfico y el pasivo detecta fallos reales bajo carga; se complementan.

Lección 13 — Circuit breaking

El número máximo de conexiones, de peticiones pendientes, de peticiones activas y de reintentos concurrentes hacia un cluster; al superarse, Envoy rechaza rápido en lugar de acumular carga y propagar la degradación.

Evita fallos en cascada y protege al servicio saturado, porque no acumula peticiones que ya no se podrían atender a tiempo.

Lección 14 — Reintentos y timeouts

Configurar una política de reintentos (condiciones, número máximo y timeout por intento) y limitar su presupuesto, porque reintentar sin control multiplica la carga sobre un servicio ya degradado y puede duplicar operaciones no idempotentes.

Reintentos de muchos clientes que multiplican la carga sobre un servicio degradado; se evita limitando el presupuesto de reintentos y usando circuit breaking.

Lección 15 — Limitación de tasa

Local, mediante un filtro que aplica un token bucket en cada instancia sin dependencias externas, y global, que consulta un servicio de rate limit compartido para aplicar un límite común entre todas las instancias.

Cuando se necesita un límite común y exacto entre varias instancias, en lugar de un límite aproximado por cada una.

Módulo 4: Seguridad y operación

Lección 16 — TLS y mTLS con SDS

Mediante transport sockets TLS en listeners (hacia los clientes) y clusters (hacia los destinos), con validación de certificados y mTLS cuando se requiere; con SDS los certificados se entregan y rotan dinámicamente sin reiniciar el proxy.

Autenticación mutua entre servicios (identidad de ambos extremos) además del cifrado del tráfico.

Lección 17 — Autenticación y autorización

jwt_authn valida tokens JWT, ext_authz consulta a un servicio externo para decidir si se permite la petición y el filtro RBAC aplica reglas de acceso locales según identidad, cabeceras o ruta.

Permite centralizar la decisión de acceso en un servicio externo con políticas propias, en lugar de repetirlas en cada aplicación.

Lección 18 — Observabilidad

Estadísticas detalladas (por listener, cluster y ruta, exportables a Prometheus), registros de acceso con formato configurable y trazas distribuidas (por ejemplo con OpenTelemetry) mediante la propagación de cabeceras de contexto.

Una cabecera de correlación como x-request-id o las de trazado (traceparent), propagadas entre servicios.

Lección 19 — Extensibilidad: Lua y Wasm

Con filtros nativos, scripts Lua para lógica ligera por petición y módulos WebAssembly (Wasm) que se cargan en caliente en un entorno aislado; cuanto más compleja la extensión, más importa su coste de rendimiento y su seguridad.

Para lógica sencilla y ligera por petición, rápida de escribir; Wasm es mejor para extensiones más complejas o reutilizables.

Lección 20 — Proyecto final: Envoy listo para producción

Configuración dinámica con un plano de control, mTLS con SDS, health checks más outlier detection, circuit breaking y reintentos acotados, timeouts explícitos, rate limiting, administración protegida, métricas, logs y trazas con alertas y una versión de Envoy actualizada.

Timeouts y reintentos acotados, circuit breaking y outlier detection, y observabilidad con alertas (más mTLS y administración protegida).